

توثيق إضافة مساعد اختيار الأحذية إلى Nike Store

دمج Jotform AI Agent داخل صفحة Homepage

نوع الوثيقة: توثيق أكاديمي وتقني

المشروع: Nike Store

الواجهة الأمامية: React + TypeScript + Vite + Chakra UI

مصدر المساعد: Jotform AI Agent

المستودع: [Nike-Store-Full-Stuck 1](#)

الملخص

يشرح هذا المستند عملية إضافة مساعد تفاعلي لاختيار أحذية Nike إلى الصفحة الرئيسية في مشروع Nike Store. كان المساعد منشوراً مسبقاً على منصة Jotform، وقد زوّد صاحب المشروع بكود التضمين الرسمي الذي يتكون من `iframe` وسكربت خارجي من Jotform. بدلاً من نسخ الكود مباشرة إلى ملف HTML عام، تم تحويله إلى مكون React مستقل حتى يصبح قابلاً للصيانة، وقابلاً لإعادة الاستخدام.

تمت المحافظة على واجهات المتجر الموجودة، ولم يتم تعديل Dashboard أو نظام Authentication أو Backend؛ لأن وظيفة المساعد تعتمد على خدمة خارجية وتُعرض داخل المتصفح من خلال `iframe`. التغيير الوظيفي اقتصر على ملفين في Frontend: إنشاء المكون `JotformShoeAssistant.tsx`، وتعديل `Landing.tsx` لاستدعائه داخل Homepage. كما تم اختبار رابط Jotform الصحيح، واكتشاف خطأ في معرف النسخة الأولية وتصحيحه قبل اعتماد الدمج.

النتيجة النهائية: يظهر قسم **Shoe Assistant** داخل Homepage بعد قسم الخدمات وقبل **Welcome Back**، وتعمل داخله واجهة Jotform التي تحتوي على **Chat** و **Voice** و **History** وحقل التفاعل مع المساعد.

الفصل الأول: تعريف المتطلب

1.1 المتطلب الوظيفي

المطلوب هو إضافة مساعد يساعد زائر المتجر في اختيار الحذاء الأنسب له. يجب أن يظهر المساعد في صفحة Homepage تحديداً، وأن يحافظ على تصميم الموقع، وأن يكون مستقلاً عن صفحات Dashboard وبقية المسارات. كما يجب أن يستطيع المستخدم التفاعل معه من خلال خدمة Jotform الجاهزة دون أن نعيد بناء نموذج الذكاء الاصطناعي داخل Django أو React.

1.2 كود التضمين الذي تم تزويده

الكود الذي زوّد به صاحب المشروع يعتمد على عنصر `iframe`:

```
<iframe
  id="JotFormIFrame-01a01f16f79070008b548ce63c5f97237f3b"
  title="مساعد اختيار أحذية Ula:"
  allowtransparency="true"
  allow="geolocation; microphone; camera; fullscreen"
  src="https://eu.jotform.com/agent/01a01f16f79070008b548ce63c5f97237f3b?"
  frameborder="0"
  style="max-width:100%; height:688px; border:none; width:100%;"
  scrolling="no">
</iframe>
```

ويُحمّل الكود سكربت Jotform المسؤول عن تهيئة وسلوك الـ `iframe`:

```
<script src="https://cdn.jotform.com/s/umd/76e215718e2/for-form-embed-handler.js">
</script>
window.jotformEmbedHandler(
  "iframe[id='JotFormIFrame-01a01f16f79070008b548ce63c5f97237f3b']",
  "https://eu.jotform.com"
);
</script>
```

هذا الكود مناسب لصفحة HTML ثابتة، ولكن في مشروع React من الأفضل تغليفه بمكوّن، وتحميل السكربت عبر دورة حياة React، وعدم وضع منطق تشغيل JavaScript خارجي داخل `index.html` دون حاجة.

الفصل الثاني: تحليل المشروع قبل التعديل

2.1 موقع Homepage

بعد فحص بنية المستودع، تبين أن المسار الرئيسي للواجهة الأمامية هو:

```
client/
├── src/
│   ├── Pages/
│   │   └── HomePage.tsx
│   ├── components/
│   │   └── Landing/
│   │       └── Landing.tsx
```

ملف `HomePage.tsx` لا يحتوي كل محتوى الصفحة، بل يستدعي مكوّن `Landing`:

```
import { Container } from "@chakra-ui/react";
import Landing from "@components/Landing/Landing";

const HomePage = () => {
  return (
    <
      <Container>
        <Landing />
      </Container>
    </>
  );
};

export default HomePage;
```

وبذلك فإن المكان الصحيح لإضافة قسم جديد داخل `Homepage` هو `Landing.tsx` ، وليس `HomePage.tsx` أو `main.tsx`. هذا الاختيار يحافظ على تنظيم المشروع؛ إذ إن `HomePage` يحدد الغلاف العام، بينما `Landing` يحدد تسلسل أقسام الصفحة.

2.2 ترتيب Homepage قبل الإضافة

كان ترتيب `Landing.tsx` كما يلي:

Hero / NIKE Collection



Products



About



Reviews



Services



Welcome Back

لذلك تم وضع المساعد بعد Services، لأنه ميزة مساعدة مرتبطة بالخدمات وتجربة التسوق، وقبل Welcome Back الموجود في نهاية محتوى Landing.

2.3 لماذا لم يتم وضعه في Backend؟

المساعد نفسه مستضاف على Jotform. وظيفة مشروع Nike Store هي تضمين واجهته، لا إعادة تنفيذ خوارزمية المحادثة. لذلك لم تكن هناك حاجة لإضافة Model أو Serializer أو View أو Migration في Django. وبقي Backend دون تغيير في هذه الإضافة.

هذا الفصل بين المسؤوليات مهم أكاديمياً؛ فالواجهة تعرض خدمة خارجية، بينما Backend الخاص بالمتجر يظل مسؤولاً عن المنتجات والطلبات والمستخدمين والمخزون والصلاحيات.

الفصل الثالث: الملفات والمجلدات المتأثرة

3.1 الملف الجديد

تم إنشاء مجلد ومكوّن مستقلين:

```
client/src/components/JotformShoeAssistant/  
└─ JotformShoeAssistant.tsx
```

العنصر	الحالة	الغرض
/JotformShoeAssistant	مجلد جديد	عزل ميزة المساعد عن بقية Components
JotformShoeAssistant.tsx	ملف جديد	عرض iframe وتحميل سكربت Jotform

3.2 الملف المعدّل

تم تعديل:

```
client/src/components/Landing/Landing.tsx
```

التعديل يتكون من إضافة Import واستدعاء Component:

```
import JotformShoeAssistant from
  "../JotformShoeAssistant/JotformShoeAssistant";
```

ثم داخل JSX:

```
<Services />
<JotformShoeAssistant />
<WelcomeBack />
```

3.3 ملفات لم يتم تعديلها

لم يتم تعديل الملفات التالية، لأن الميزة لا تحتاج إليها:

الجزء	سبب عدم التعديل
server/store/models.py	لا توجد بيانات جديدة محفوظة في قاعدة البيانات
server/store/serializers.py	لا يوجد JSON جديد يمر عبر Django
server/store/views.py	Jotform يعالج المحادثة خارج Backend المشروع
server/store/urls.py	لا يوجد Endpoint جديد
/client/src/api	لا يوجد طلب إلى API المتجر
/client/src/auth	المساعد متاح للزائر ولا يحتاج Login
/client/src/Admin	المطلوب Homepage فقط
client/src/Routes/Routes.tsx	لا يوجد Route جديد
client/index.html	السكربت يُدار من المكوّن وليس من HTML العام

الفصل الرابع: تصميم المكوّن الجديد

4.1 الثوابت المركزية

في بداية المكوّن تم تعريف القيم التي يحتاجها التكامل:

```
const JOTFORM_IFRAME_ID =
  "JotFormIFrame-01a01f16f79070008b548ce63c5f97237f3b";

const JOTFORM_ORIGIN = "https://eu.jotform.com";

const JOTFORM_SCRIPT_SRC =
  "https://cdn.jotfor.ms/s/umd/76e215718e2/for-form-embed-handler.js";
```

وضع القيم في ثوابت يجعل الكود أوضح ويمنع تكرار المعرّف والرابط في أكثر من مكان. كما يسهّل تحديث المساعد مستقبلاً إذا تغيّر Agent ID.

4.2 تعريف TypeScript للدالة الخارجية

الدالة `jotformEmbedHandler` لا تأتي من TypeScript، لأنها تُضاف إلى `window` عند تحميل السكريبت خارجي. لذلك تمت إضافة تعريف `global`:

```
declare global {  
  interface Window {  
    jotformEmbedHandler?: (  
      selector: string,  
      origin: string,  
    ) => void;  
  }  
}
```

العلامة `?` تعني أن الدالة اختيارية؛ فقد لا تكون موجودة لحظة تنفيذ الكود قبل تحميل السكريبت. وهذا يمنع TypeScript من إظهار خطأ عند الوصول إليها بطريقة آمنة.

4.3 استخدام `useEffect`

يجب تحميل السكريبت بعد أن يصبح المكوّن جزءاً من DOM. لهذا السبب استخدم `useEffect`:

```
useEffect(() => {  
  const runEmbedHandler = () => {  
    window.jotformEmbedHandler?.(  
      `iframe[id='${JOTFORM_IFRAME_ID}']`,  
      JOTFORM_ORIGIN,  
    );  
  };  
  
  // script loading logic  
, []);
```

المصفوفة الفارغة `[]` تعني أن العملية تُنفذ عند Mount مرة واحدة، وليس في كل إعادة Render. وهذا يمنع إضافة السكريبت مراراً.

4.4 منع تكرار السكربت

```
const existingScript = document.querySelector<HTMLScriptElement>(`script[src="${JOTFORM_SCRIPT_SRC}"]`,
);

if (existingScript) {
  runEmbedHandler();
  return;
}
```

قبل إنشاء Script جديد، يبحث المكوّن عن Script موجود بالمصدر نفسه. إذا كان موجوداً، يعاد تشغيل Handler فقط. هذه الطريقة مفيدة إذا ظهر المكوّن أكثر من مرة أو في بيئة تطوير فيها Hot Reload.

4.5 إنشاء عنصر Script

```
const script = document.createElement("script");
script.src = JOTFORM_SCRIPT_SRC;
script.async = true;
script.onload = runEmbedHandler;
document.body.appendChild(script);
scriptRef.current = script;
```

الخطوات هي:

1. إنشاء عنصر `script`.
2. وضع عنوان مكتبة Jotform فيه.
3. تفعيل التحميل غير المتزامن `async`.
4. تشغيل `runEmbedHandler` بعد نجاح التحميل.
5. إدخال العنصر في `document.body`.
6. حفظ مرجعه داخل `useRef` لإزالته عند الحاجة.

4.6 تنظيف Script عند إزالة المكوّن

```
return () => {  
  scriptRef.current?.remove();  
  scriptRef.current = null;  
};
```

دالة التنظيف تعمل عند Unmount. وهي تمنع ترك Script ديناميكي في الصفحة إذا غادر المستخدم Homepage أو أعيد تركيب المكوّن.

الفصل الخامس: بناء واجهة iframe

5.1 الغلاف الخارجي

```
<Box  
  as="section"  
  aria-labelledby="shoe-assistant-title"  
  width="100%"  
  maxW="960px"  
  marginX="auto"  
  marginTop={{ base: 12, md: 16 }}  
  marginBottom={{ base: 12, md: 16 }}  
  paddingX={{ base: 0, md: 4 }}  
>
```

استُخدم `Box` من Chakra UI حتى يتوافق المكوّن مع نظام الواجهة الموجود. `maxW` يمنع تمدد المساعد بصورة مبالغ فيها على الشاشات الواسعة، و `"marginX=auto"` يجعله في منتصف الصفحة.

5.2 العنوان والوصف

```
<Heading
  id="shoe-assistant-title"
  textAlign="center"
  fontSize={{ base: "2xl", md: "3xl" }}
  marginBottom={2}
>
  Shoe Assistant
</Heading>

<Text textAlign="center" color="gray.500" marginBottom={6}>
  Find the Nike shoes that best match your style and needs.
</Text>
```

إضافة عنوان واضح تعطي المستخدم سياقاً قبل تحميل iframe، كما أن `aria-labelledby` يربط القسم بالعنوان لأغراض Accessibility.

5.3 عنصر iframe النهائي

```
<iframe
  id={JOTFORM_IFRAME_ID}
  title="مساعد اختيار أحذية Ula:"
  src={`$${JOTFORM_ORIGIN}/agent/01a01f16f79070008b548ce63c5f97237f3b?embed`}
  allow="geolocation; microphone; camera; fullscreen"
  frameBorder="0"
  scrolling="no"
  loading="lazy"
  style={{
    display: "block",
    width: "100%",
    maxWidth: "100%",
    height: "688px",
    border: "none",
  }}
/>
```

شرح الخصائص

الخاصية	الوظيفة
id	يطابق Selector الذي يستخدمه Jotform Handler
title	وصف للـ iframe وتحسين Accessibility
src	عنوان Agent Ula مع معاملات التضمين
allow	السماح بالميزات التي أعلنها الكود الأصلي، مثل microphone و camera و fullscreen
frameBorder	إزالة الإطار التقليدي
scrolling	منع شريط تمرير داخلي غير مرغوب
"loading="lazy	تأجيل تحميل المساعد حتى يقترب المستخدم من القسم
height	الحفاظ على مساحة ثابتة مناسبة للمحادثة

5.4 سبب استخدام "loading="lazy"

المساعد موجود أسفل أجزاء متعددة من Homepage. لذلك لا حاجة لتحميله قبل أن يصل المستخدم إليه. التحميل الكسول يقلل العمل الأولي للمتصفح، مع بقاء المساعد جاهزاً عند الوصول إلى موضعه. وتظل القيمة height محجوزة حتى لا يحدث تغير مفاجئ كبير في تخطيط الصفحة.

الفصل السادس: إدراج المكوّن داخل Homepage

Import 6.1

تمت إضافة السطر التالي إلى Landing.tsx :

```
import JotformShoeAssistant from
"../JotformShoeAssistant/JotformShoeAssistant";
```

مسار الاستيراد صحيح لأن `Landing.tsx` داخل:

```
client/src/components/Landing/
```

بينما المكوّن الجديد داخل:

```
client/src/components/JotformShoeAssistant/
```

ولذلك ننتقل مستوى واحداً إلى `components` ثم ندخل المجلد الجديد.

6.2 موضع الاستدعاء

```
{/* Services */}
<Services />
{/* AI shoe-selection assistant */}
<JotformShoeAssistant />
{/* Welcome Back */}
<WelcomeBack />
```

الاستدعاء داخل JSX يجعل React ينشئ المكوّن عند عرض `Landing`. وعند ترك `Homepage`، ينفذ `React` دالة التنظيف الخاصة بـ `useEffect`.

6.3 لماذا لم نضع المكوّن في `HomePage.tsx` ؟

كان من الممكن وضعه في `HomePage.tsx` ، لكن ذلك كان سيخلط بين الغلاف العام ومحتوى `Landing`. `Landing.tsx` يحتوي أصلاً على `Products` و `About` و `Reviews` و `Services`، ولذلك هو المكان المنطقي لتسلسل قسم المساعد بصرياً.

الفصل السابع: تصحيح معرفّ Agent والتحقق من

المصدر

7.1 الخطأ الأول

أثناء التحضير الأولي، استُخدم معرف ينتهي بـ `f3b3`، فكانت النتيجة من Jotform:

Agent not found

هذا النوع من الأخطاء يعني أن رابط Agent لا يشير إلى المورد المنشور، وليس خطأ في React أو Chakra UI.

7.2 المعرف الصحيح

بعد العودة إلى كود المستخدم، تم اعتماد المعرف:

01a01f16f79070008b548ce63c5f97237f3b

وعنوان المصدر النهائي:

<https://eu.jotform.com/agent/01a01f16f79070008b548ce63c5f97237f3b?embedMo>

7.3 نتيجة التحقق

عند فتح الرابط الصحيح، أصبح عنوان الصفحة `Ula` بدلاً من `Agent not found`. وعند فتح Homepage المحلي، ظهرت عناصر Jotform التفاعلية:

AI
AI Terms
Chat
Voice
History
Chat With AI Agent
Type here

وهذا يؤكد أن ال iframe والسكربت تم تحميلهما، وأن القسم ليس مجرد مساحة فارغة أو رابطاً غير صالح.

الفصل الثامن: تدفق التشغيل الكامل

8.1 من تشغيل الموقع إلى ظهور المساعد

```
Vite تشغيل
↓
React application تحميل
↓
Homepage الخاص بـ Route فتح
↓
Landing يستدعي HomePage
↓
Landing يرسم الأقسام بالترتيب
↓
JotformShoeAssistant يرسم Landing
↓
DOM يُضاف إلى iframe
↓
Jotform يبحث عن سكريبت useEffect
↓
async تحميل السكريبت بشكل
↓
jotformEmbedHandler يستدعي script.onload
↓
iframe يهيئ Jotform
↓
المستخدم يبدأ المحادثة
```

8.2 هل يمر الطلب عبر Django؟

لا يمر تفاعل المحادثة عبر Django Backend الخاص بالمتجر. المتصفح يتصل بخدمة Jotform من خلال iframe، بينما يبقى Django مسؤولاً عن API المتجر فقط. هذا يعني أن إضافة المساعد لا تغيّر قواعد المنتجات أو السلة أو المستخدمين.

8.3 هل يحتاج المستخدم إلى Login؟

لا. المكوّن موجود في Homepage العامة، ولا يحتوي على شرط `isAuthenticated`. يستطيع الزائر استخدامه قبل إنشاء حساب. وإذا كانت إعدادات Agent الداخلية تتطلب معلومات معينة، فهي تُدار من Jotform

الفصل التاسع: الاختبارات المنفذة

9.1 اختبار البناء

تم تشغيل:

```
cd client  
npm run build
```

والنتيجة:

```
✓ built in 1.90s
```

مرّ فحص TypeScript و Vite بنجاح. ظهر تحذير متعلق بحجم حزمة JavaScript بعد التصغير، لكنه تحذير موجود على مستوى حجم التطبيق وليس فشلاً في مكوّن Jotform.

9.2 اختبار Homepage محلياً

تم تشغيل خادم التطوير:

```
npm run dev -- --host 0.0.0.0
```

ثم فتح:

```
http://127.0.0.1:5173/
```

ظهرت Homepage وبقي ترتيب المتجر سليماً، ثم تم الانتقال إلى موضع المساعد. ظهر العنوان:

Shoe Assistant

وظهرت واجهة Jotform وعناصر Chat و Voice و History وحقل كتابة الرسائل.

9.3 جدول الاختبارات

الاختبار	النتيجة
TypeScript compilation	ناجح
Vite production build	ناجح
فتح Homepage	ناجح
ظهور Products و About و Reviews	مستمر دون تغيير
ظهور Services	مستمر دون تغيير
ظهور Shoe Assistant	ناجح
تحميل Agent الصحيح	ناجح
ظهور Chat و Voice و History	ناجح
تعديل Dashboard	لم يحدث
تعديل Django Backend	لم يحدث
إضافة سكرت مكرر	يمنعها فحص <code>existingScript</code>

الفصل العاشر: اعتبارات الأمان والخصوصية

10.1 مصدر خارجي موثوق

الإضافة تستخدم رابطاً خارجياً من نطاق Jotform ونطاق CDN الخاص به. يجب إبقاء المصدر معروفاً ومراجعة أي تغيير في كود التضمين قبل اعتماده. لا يتم تنفيذ JavaScript مكتوب من إدخال المستخدم؛ بل يتم تحميل

سكربت ثابت من العنوان الذي زود به صاحب المشروع.

10.2 صلاحيات iframe

تم الإبقاء على الصلاحيات التي وردت في كود التضمين الأصلي:

```
geolocation; microphone; camera; fullscreen
```

إذا لم يكن Voice أو Camera مطلوباً مستقبلاً، يمكن تقليل هذه الصلاحيات إلى الحد الأدنى. تقليل الصلاحيات مبدأً آمناً جيد، لأن iframe لا ينبغي أن يطلب قدرة لا يحتاج إليها.

10.3 عدم تسريب Token المتجر

المكوّن لا يستدعي `apiRequest` ولا يقرأ Local Storage ولا يرسل Token المتجر إلى Jotform. لذلك يبقى Token الخاص بـ Django منفصلاً عن جلسة Jotform.

10.4 سياسة الخصوصية

بما أن المستخدم قد يكتب بيانات داخل مساعد خارجي، ينبغي أن توضح سياسة الخصوصية في النسخة المنشورة أن المحادثة تتم عبر Jotform وفق سياسات الخدمة الخارجية. هذه نقطة مهمة عند نشر المشروع بشكل فعلي أو استخدامه مع عملاء حقيقيين.

الفصل الحادي عشر: طريقة تثبيت التعديل على جهاز آخر

11.1 الحصول على النسخة

```
git clone https://github.com/Karam-Ghanem/Nike-Store-Full-Stuck.git
cd Nike-Store-Full-Stuck
```

11.2 تثبيت Frontend وتشغيله

```
cd client
npm ci
npm run dev
```

ثم يفتح المستخدم:

```
http://localhost:5173
```

11.3 التأكد من الملفات

يجب أن يكون الملف الجديد موجوداً في:

```
client/src/components/JotformShoeAssistant/JotformShoeAssistant.tsx
```

ويجب أن يحتوي `Landing.tsx` على:

```
import JotformShoeAssistant from
  "../JotformShoeAssistant/JotformShoeAssistant";
```

و:

```
<JotformShoeAssistant />
```

11.4 في حال تغيير Agent لاحقاً

إذا أنشأ صاحب المشروع Agent جديداً، يجب تحديث ثلاثة مواضع مترابطة داخل المكوّن:

القيمة	مكانها
JOTFORM_IFRAME_ID	الثابت الخاص بالـ iframe
src داخل Agent ID	رابط iframe
runEmbedHandler داخل Selector	استدعاء Jotform Handler

ويجب اختبار الرابط الجديد مباشرة قبل استخدامه في الموقع.

الفصل الثاني عشر: الفرق بين نسخ HTML ونسخة React

12.1 الطريقة المباشرة

في HTML ثابت، يمكن وضع:

```
<iframe></iframe>
<script></script>
```

داخل الملف مباشرة. هذه الطريقة قصيرة، لكنها لا تستفيد من دورة حياة React ولا تمنع بالضرورة تكرار Script عند إعادة التركيب.

12.2 الطريقة المعتمدة في المشروع

في React تم تقسيم العملية إلى:

```
iframe مسؤول عن JSX
Script مسؤول عن useEffect
مسؤول عن مرجع التنظيف useRef
مسؤول عن مكان الظهور Landing
```

هذا الفصل يجعل التغيير قابلاً للفهم والاختبار. كما أن Component مستقل يمكن نقله إلى صفحة أخرى مستقبلاً دون نسخ الكود.

الفصل الثالث عشر: الأثر على بنية النظام

13.1 المعمارية قبل الإضافة

```
React Router
  ↓
HomePage
  ↓
Landing
  ├── Products
  ├── About
  ├── Reviews
  ├── Services
  └── Welcome Back
```

13.2 المعمارية بعد الإضافة

```
React Router
  ↓
HomePage
  ↓
Landing
  ├── Products
  ├── About
  ├── Reviews
  ├── Services
  ├── JotformShoeAssistant
  │   ├── iframe
  │   └── Jotform Embed Handler
  └── Welcome Back
```

13.3 مصدر البيانات

البيانات	مصدرها
Products	Django REST API وقاعدة البيانات
Cart	Django REST API للمستخدم المسجل
Orders	Django REST API
Authentication	Django Token Authentication
Shoe Assistant UI	Jotform iframe
محادثة المساعد	خدمة Jotform الخارجية

الفصل الرابع عشر: الخلاصة

تمت إضافة مساعد اختيار الأحذية بنجاح إلى Homepage دون تغيير تصميم Dashboard أو Backend أو Authentication. الحل النهائي يتبع نمط React الصحيح عبر إنشاء Component مستقل، وتحميل سكربت Jotform عند Mount، وتشغيل Handler بعد اكتمال التحميل، ثم تنظيف العنصر عند Unmount.

من الناحية البرمجية، التغيير صغير لكنه منظم: مجلد جديد وملف مكوّن جديد، إضافة إلى تعديل واحد في Landing.tsx. ومن الناحية المعمارية، بقيت حدود النظام واضحة؛ ف Django يدير المتجر، و Jotform يدير المساعد، و React يربط تجربة العرض بينهما.

أثبت الاختبار أن رابط Agent الصحيح يعمل، وأن Homepage تُبنى دون أخطاء، وأن عناصر Chat و Voice و History تظهر داخل الصفحة. كما تم توثيق خطأ المعرّف الأول وتصحيحه، حتى يستطيع فريق المشروع فهم سبب المشكلة إذا ظهرت مستقبلاً.

التوصية التشغيلية: قبل نشر المشروع، يجب التأكد من إعدادات الخصوصية في Jotform، واختبار المساعد على الهاتف، ومراجعة صلاحيات camera و microphone، ثم تنفيذ `npm run build` في بيئة الإنتاج.

الملحق أ: الملف الكامل للمكوّن الجديد

```
import { Box, Heading, Text } from "@chakra-ui/react";
import { useEffect, useRef } from "react";

declare global {
  interface Window {
    jotformEmbedHandler?: (
      selector: string,
      origin: string,
    ) => void;
  }
}

const JOTFORM_IFRAME_ID =
  "JotFormIFrame-01a01f16f79070008b548ce63c5f97237f3b";
const JOTFORM_ORIGIN = "https://eu.jotform.com";
const JOTFORM_SCRIPT_SRC =
  "https://cdn.jotfor.ms/s/umd/76e215718e2/for-form-embed-handler.js";

const JotformShoeAssistant = () => {
  const scriptRef = useRef<HTMLScriptElement | null>(null);

  useEffect(() => {
    const runEmbedHandler = () => {
      window.jotformEmbedHandler?.(
        `iframe[id='${JOTFORM_IFRAME_ID}']`,
        JOTFORM_ORIGIN,
      );
    };

    const existingScript = document.querySelector<HTMLScriptElement>(
      `script[src='${JOTFORM_SCRIPT_SRC}']`,
    );

    if (existingScript) {
      runEmbedHandler();
      return;
    }

    const script = document.createElement("script");
    script.src = JOTFORM_SCRIPT_SRC;
```

```

script.async = true;
script.onload = runEmbedHandler;
document.body.appendChild(script);
scriptRef.current = script;

return () => {
  scriptRef.current?.remove();
  scriptRef.current = null;
};
}, []);

return (
  <Box
    as="section"
    aria-labelledby="shoe-assistant-title"
    width="100%"
    maxW="960px"
    marginX="auto"
    marginTop={{ base: 12, md: 16 }}
    marginBottom={{ base: 12, md: 16 }}
    paddingX={{ base: 0, md: 4 }}
  >
    <Heading
      id="shoe-assistant-title"
      textAlign="center"
      fontSize={{ base: "2xl", md: "3xl" }}
      marginBottom={2}
    >
      Shoe Assistant
    </Heading>
    <Text textAlign="center" color="gray.500" marginBottom={6}>
      Find the Nike shoes that best match your style and needs.
    </Text>
    <Box
      width="100%"
      minH="688px"
      overflow="hidden"
      borderRadius="lg"
      background="transparent"
    >
      <iframe
        id={JOTFORM_IFRAME_ID}
        title="Ula: مساعد اختيار أحذية"
        src={`$${JOTFORM_ORIGIN}/agent/01a01f16f79070008b548ce63c5f97237`

```



```

allow="geolocation; microphone; camera; fullscreen"
frameBorder="0"
scrolling="no"
loading="lazy"
style={{
  display: "block",
  width: "100%",
  maxWidth: "100%",
  height: "688px",
  border: "none",
}}
/>
</Box>
</Box>
);
};

export default JotformShoeAssistant;

```

الملحق ب: الفرق الفعلي في Landing.tsx

```

import WelcomeBack from "../WelcomeBack/WelcomeBack";
import MainTitle from "../PublicComponentents/MainTitle";
+import JotformShoeAssistant from "../JotformShoeAssistant/JotformShoeAss:
@@
    <Services />
+    <JotformShoeAssistant />
    <WelcomeBack />

```

المراجع